RK816 Developer Guide

ID: RK-KF-YF-67

Release Version: V1.0.1

Release Date: 2022-01-06

Security Level: □Top-Secret □Secret □Internal ■Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED “AS IS”. ROCKCHIP ELECTRONICS CO., LTD. (“ROCKCHIP”) DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS, MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip’s registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2022. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, No.89, Software Boulevard, TongPan Rd, Fuzhou, Fujian, PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

This document primarily introduces the various sub-modules of the RK816, covering related concepts, functions, DTS configuration, and analysis and troubleshooting of common issues.

Product Versions

Chip Models	Kernel Versions
RK816	3.10, 4.4, 4.19

Target Audience

This document (this guide) is primarily intended for the following engineers:

- Technical Support Engineers
- Software Development Engineers

Revision History

Version	Author	Date	Modification Description
V1.0.0	Zhang Qing	2019-11-25	Initial version
V1.0.1	Huang Ying	2022-05-30	Format modification

Table of Contents

RK816 Developer Guide
- 1. Basics
  - 1.1 Overview
  - 1.2 Functions
  - 1.3 Chip Pin Functions
  - 1.4 Key Concepts
  - 1.5 Power-On Conditions and Sequence
- 2. Configuration
  - 2.1 Driver and menuconfig
  - 2.2 DTS Configuration
  - 2.3 Function Interfaces
- 3. Debug
  - 3.1 Kernel 3.10
  - 3.2 4.4 Kernel
  - 3.3 4.19 Kernel

1. Basics

1.1 Overview

The RK816 is a high-performance PMIC integrating four high-current DCDC converters, one BOOST converter, six LDOs, one OTG output, one RTC, adjustable power-on sequencing, switched-mode charging, intelligent power path management, and a coulomb counter.

Power rails in the system are generally categorized into two types: DCDC and LDO. The general characteristics of these two power types are as follows (please search for detailed information independently):

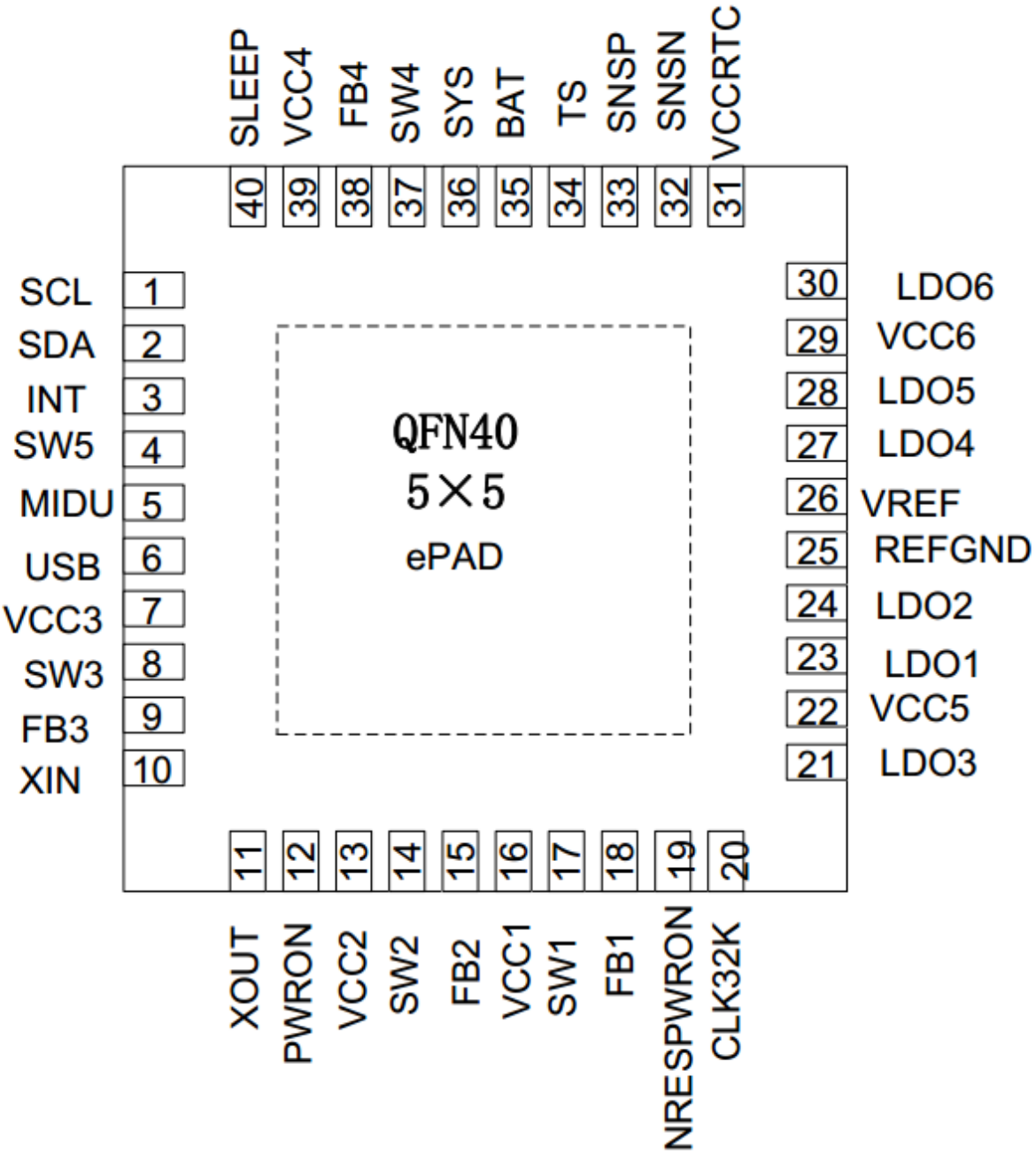
1. DCDC: Offers high efficiency when there is a large difference between input and output voltage, but suffers from relatively large ripple and higher cost. Therefore, it is used for applications with large voltage differentials and high current loads. It generally operates in two modes: PWM mode, which provides good ripple transient response but lower efficiency; and PFM mode, which offers high efficiency but poor load capability.
2. LDO: Exhibits low efficiency and low cost when there is a large difference between input and output voltage. To improve LDO conversion efficiency, the system implements relevant optimizations. For example, if the LDO output voltage is 1.1V, its input voltage can be supplied by the VCCIO_3.3V DCDC to enhance efficiency. Therefore, if the circuit design permits, LDOs should be connected to the DCDC output rail as much as possible, while paying attention to the power-on sequencing.

1.2 Functions

From the user's perspective, the functions of the RK816 can be summarized into five parts:

1. Regulator function: Controls the power status of each DCDC and LDO rail.
2. RTC function: Provides clock timing and alarm functions.
3. GPIO function: Features two push-pull output pins (out1 and out2, output-only) that can be used as general-purpose GPIOs.
4. PWRKEY function: Detects power button press and release events, saving one GPIO pin for the AP.
5. Charging and fuel gauge functions are not detailed in this document; please refer to "Rockchip_RK818_RK816_Developer_Guide_Fuel_Gauge_EN" for comprehensive information.

1.3 Chip Pin Functions



In the following description, special attention should be paid to the SLEEP and INT pins:

PIN NO	PIN NAME	PIN DESCRIPTION
1	SCL	I2C clock input
2	SDA	I2C data input and output
3	INT	Interrupt request pin.

Copyright ©2019 Fuzhou Rockchip Electronics Co., Ltd.

PIN NO	PIN NAME	PIN DESCRIPTION
4	SW5	Charger switching node / boost switching node
5	MIDU	Middle point of USB power supply / boost output
6	USB	USB power supply
7	VCC3	Power supply of buck3
8	SW3	Switching node of buck3
9	VFB3	Output feedback voltage of buck3
10	XIN	32.768KHz crystal oscillator input
11	XOUT	32.768KHz crystal oscillator output
12	PWRON	Power on or power off enable pin, active low, internal 20k pull high to power supply
13	VCC2	Power supply of buck2
14	SW2	Switching node of buck2
15	VFB2	Output feedback voltage of buck2
16	VCC1	Power supply of buck1
17	SW1	Switching node of buck1
18	VFB1	Output feedback voltage of buck1
19	NRESPWON	Reset pin after power on, active low.

20	CLK32K	32.768KHz clock output, open drain
21	LDO3	LDO3 output
22	VCC5	Power supply of LDO1/2/3
23	LDO1	LDO1 output
24	LDO2	LDO2 output
25	REFGND	Reference ground
26	VREF	Internal reference voltage
27	LDO4	LDO4 output
28	LDO5	LDO5 output
29	VCC6	Power supply of LDO4/5/6
30	LDO6	LDO6 output
31	VCCRTC	Power supply of RTC, decouple it to GND with a 1uF cap
32	SNSN	Battery charging or discharging current sense negative terminal
33	SNSP	Battery charging or discharging current sense postive terminal
34	TS(GPIO1)	Thermistor input. Connect a thermistor from this pin to ground. The thermistor is usually inside the battery pack. (multi-function for GPIO1)
35	BAT	Battery positive terminal
36	SYS	System terminal
37	SW4	Switching node of buck4
38	VFB4	Output feedback voltage of buck4
39	VCC4	Power supply of buck4
40	SLEEP	Sleep mode control input
Exposed pad	Exposed ground	Ground

1.4 Key Concepts

- I2C Address

7-bit slave address: 0x1a

- The PMIC has three operating modes

1. PMIC Normal Mode

When the system is running normally, the PMIC operates in normal mode, with the pmic_sleep pin held low.

2. PMIC Sleep Mode

To minimize standby power consumption during system suspend, the PMIC switches to sleep mode. In this mode, output voltages for certain rails are typically reduced or outputs are disabled entirely, configurable based on specific product requirements. During system standby, the AP configures the PMIC for sleep mode via an I2C command and then drives the pmic_sleep pin high to enter the sleep state. Upon SoC wakeup, the pmic_sleep pin returns to low, causing the PMIC to exit sleep mode.

3. PMIC Shutdown Mode

When the system initiates the shutdown sequence, the PMIC must power down the entire system. The AP configures the PMIC for shutdown mode via an I2C command and then drives the pmic_sleep pin high to enter the shutdown state.

- pmic_sleep Pin

Normally held low, indicating the PMIC is in normal mode. Driving this pin high switches the PMIC to either sleep or shutdown mode.

- pmic_int Pin

Normally held high. It transitions to low when an Interrupt occurs. If the Interrupt remains unhandled, the pin stays low.

- out1/out2 Pins

These pins can function as general-purpose GPIOs (push-pull output) but support only GPIO output mode.

- pmic_pwron Pin

To implement pwrkey functionality, the hardware must connect the power button to this pin. The Driver uses this pin to detect button press and release events.

- Operating Modes of DCDC Rails

  The DCDC supports PWM (also known as force PWM) and PFM modes. However, the PMIC features a mode that dynamically switches between PWM and PFM, commonly referred to as AUTO mode. The PMIC supports two configurations: PWM mode and AUTO PWM/PFM mode. While AUTO mode offers higher efficiency, it exhibits poorer ripple and transient response. For system stability, the device is configured to operate in PWM mode during runtime and switches to AUTO PWM/PFM mode when the system enters sleep state.
- DCDC3 Voltage Regulation

The DCDC3 power rail is unique; its voltage cannot be modified via Register settings and must be adjusted using external voltage divider resistors. Therefore, if a voltage change is required, the peripheral hardware must be modified. In Rockchip solutions, this rail is typically used as VCC_DDR.

- Runtime Voltage Adjustment Range for DCDC and LDO

1. Discontinuous DCDC Voltage Range:

Voltage Range (V)	Step Value (mV)	Specific Voltage Levels (V)
0.7125 ~ 1.45	12.5	0.7125, 0.725, 0.7375, ..., 1.45
1.8 ~ 2.2	200	1.8, 2.0, 2.2
2.3	N/A	2.3

2. Continuous LDO Voltage Range:

Voltage Range (V)	Step Value (mV)	Specific Voltage Levels (V)
0.8 ~ 3.4	100	0.8, 0.9, 1.0, 1.1, 1.2, ..., 3.4

1.5 Power-On Conditions and Sequence

1. Power-On Conditions

The PMIC can be powered on if any of the following conditions are met:

- The EN signal transitions from low to high level.
- The EN signal remains at high level, and an RTC alarm Interrupt is triggered.
- The EN signal remains at high level, and the PWRON button is pressed.

2. Power-On Sequence

Power-on sequence requirements for each power rail may vary across different SoC platforms. The current power-on sequences are listed below; please refer to the latest datasheet for specific details:

BOOT(OTP)			1（RK816B-1）		0（RK816B-2）		0（RK816B-3）	
	Output voltage range	Rate Current	Default voltage	Power up sequence	Default voltage	Power up sequence	Default voltage	Power up sequence
BUCK1	0.7125V-2.3V（0.7125~1.45V，step 12.5mV)	2A	1.1V	2	1.0V	3	1.1V	2
BUCK2	0.7125V-2.3V（0.7125~1.45V，step 12.5mV)	2A	1.1V	1	2.2V	1	1.1V	1
BUCK3	setting by external resistors	1A	x	3	x	4	x	3
BUCK4	0.8V-3.5V(step 0.1V)	1A	3.3V	1	3.3V	6	3.3V	1
BOOST	4.7-5.4V(step 0.1V)	2A	5V	OFF	5V	OFF	5V	OFF
LDO1	0.8V-3.4V(step 0.1V)	300mA	1.0V	OFF	1.0V	2	1.8V	OFF
LDO2	0.8V-3.4V(step 0.1V)	300mA	1.8V	1	1.8V	5	1.8V	1
LDO3	0.8V-3.4V(step 0.1V)	100mA	1.1V	1	1.0V	2	1.1V	1
LDO4	0.8V-3.4V(step 0.1V)	300mA	1.0V	OFF	3.3V	6	3.3V	OFF
LDO5	0.8V-3.4V(step 0.1V)	300mA	3.0V	4	1.8V	OFF	3.3V	4
LDO6	0.8V-3.4V(step 0.1V)	300mA	3.0V	4	2.8V	OFF	3.3V	4

2. Configuration

2.1 Driver and menuconfig

3.10 Kernel Configuration

RK816 driver file:

```
drivers/mfd/rk816.c
drivers/input/misc/rk816-pwrkey.c
drivers/rtc/rtc-rk816.c
drivers/gpio/gpio-rk816.c
drivers/regulator/rk816-regulator.c
drivers/power/rk816_battery.c
```

For the RK816 DTS file, refer to:

```
arch/arm/boot/dts/rk816.dtsi
arch/arm/boot/dts/rk3126-86v-rk816.dts
```

Macro configuration corresponding to menuconfig:

```
CONFIG_MFD_RK816
CONFIG_GPIO_RK816
CONFIG_RTC_RK816
CONFIG_REGULATOR_RK816
CONFIG_INPUT_RK816_PWRKEY
CONFIG_BATTERY_RK816
```

4.4 Kernel Configuration

RK816 Driver Files:

```
drivers/mfd/rk808.c
drivers/input/misc/rk8xx-pwrkey.c
drivers/rtc/rtc-rk808.c
drivers/gpio/gpio-rk8xx.c
drivers/regulator/rk808-regulator.c
drivers/clk/clk-rk808.c
drivers/power/rk816_battery.c
```

Macro configuration corresponding to menuconfig:

```
CONFIG_MFD_RK808
CONFIG_RTC_RK808
CONFIG_GPIO_RK8XX
CONFIG_REGULATOR_RK808
CONFIG_INPUT_RK8XX_PWRKEY
CONFIG_COMMON_CLK_RK808
CONFIG_BATTERY_RK816
```

4.19 Kernel Configuration

RK816 Driver File:

```
drivers/mfd/rk808.c
drivers/input/misc/rk816-pwrkey.c       // Different from kernel 4.4
drivers/rtc/rtc-rk808.c
drivers/pinctrl/pinctrl-rk816.c         // Different from kernel 4.4
drivers/regulator/rk808-regulator.c     // Different from the 4.4 kernel
drivers/clk/clk-rk808.c
drivers/power/rk816_battery.c
```

Macro configuration corresponding to menuconfig:

```
CONFIG_MFD_RK808
CONFIG_RTC_RK808
CONFIG_PINCTRL_RK816
CONFIG_REGULATOR_RK808
CONFIG_INPUT_RK816_PWRKEY
CONFIG_COMMON_CLK_RK808
CONFIG_BATTERY_RK816
```

2.2 DTS Configuration

3.10 Kernel Configuration

The DTS configuration includes: I2C bus attachment, main device, regulator, RTC, poweroff, and other sections.

```
&i2c1 {
    rk816: rk816@1a {
        reg = <0x1a>;
        status = "okay";
    };
};

/include/ "rk816.dtsi"
&rk816 {
    gpios = <&gpio1 GPIO_A5 GPIO_ACTIVE_HIGH>, <&gpio1 GPIO_A1 GPIO_ACTIVE_LOW>;
    rk816,system-power-controller;
    rk816,support_dc_chg = <1>;/*1: dc chg; 0:usb chg*/
    io-channels = <&adc 0>;
    gpio-controller;
    #gpio-cells = <2>;
    rtc {
        status = "okay";
    };

    regulators {
        rk816_dcdc1_reg: regulator@0{
            regulator-name= "vdd_arm";
            regulator-min-microvolt = <700000>;
            regulator-max-microvolt = <1500000>;
            regulator-initial-mode = <0x1>;
```

```
            regulator-initial-state = <3>;
            regulator-always-on;
            regulator-state-mem {
                regulator-state-mode = <0x2>;
                regulator-state-disabled;
                regulator-state-uv = <900000>;
            };
        };

        rk816_dcdc2_reg: regulator@1 {
            regulator-name= "vdd_logic";
            regulator-min-microvolt = <700000>;
            regulator-max-microvolt = <1500000>;
            regulator-initial-mode = <0x1>;
            regulator-initial-state = <3>;
            regulator-always-on;
            regulator-state-mem {
                regulator-state-mode = <0x2>;
                regulator-state-enabled;
                regulator-state-uv = <1000000>;
            };
        };
        rk816_dcdc3_reg: regulator@2 {
                    ............
        };
        ..............................
    };
};
```

1. I2C Attachment

The complete rk816 node must be attached under the corresponding I2C node, with `status = "okay";` configured.

2. Main Section

- Non-modifiable parts

```
    rk816,system-power-controller: Declare that the RK816 supports system power-off
management.
    gpio-controller: Declares that the RK816 has GPIO functionality.
    #gpio-cells: The number of parameters that must be specified when referencing the RK816
GPIO.
```

Note: If a node needs to reference an RK816 GPIO, use the following format:

`gpios = <&rk816 0 GPIO_ACTIVE_LOW>;` First parameter: &rk816 is fixed and must not be changed; Second parameter: Specifies which RK816 GPIO to reference; valid values are 0 or 1, where 0 corresponds to out1 and 1 corresponds to out2; Third parameter: GPIO polarity.

- Modifiable sections

gpios: Specifies the pmic_int (first) and pmic_sleep (second) pins;

3. regulator section

- `regulator-name`: Power supply name; it is recommended to keep this consistent with the hardware schematic, as this name must match when using the regulator_get interface;
- `regulator-min-microvolt`: Minimum voltage adjustable at runtime;
- `regulator-max-microvolt`: Maximum voltage adjustable at runtime;
- `regulator-initial-mode`: DCDC operating mode at runtime; typically configured as 1. 1: force pwm, 2: auto pwm/pfm;
- `regulator-state-mode`: DCDC operating mode during suspend; typically configured as 2. 1: force pwm, 2: auto pwm/pfm;
- `regulator-initial-state`: Mode during suspend; must be configured as 3;
- `regulator-boot-on`: When this property is present, the power supply is enabled upon regulator registration;
- `regulator-always-on`: When this property is present, the power supply cannot be disabled at runtime and is enabled upon registration;
- `regulator-state-enabled`: Maintains power state during suspend; to disable this power supply, change to "regulator-state-disabled";
- `regulator-state-uv`: Standby voltage when power is maintained during suspend.

Note:

If the values of `regulator-min-microvolt` and `regulator-max-microvolt` are equal, the system framework will automatically set and enable this regulator voltage during registration without requiring user intervention.

If `regulator-boot-on` or `regulator-always-on` is present, the system framework will enable the regulator by default during registration. In this case, the regulator voltage depends on two scenarios: if `regulator-min-microvolt` and `regulator-max-microvolt` are equal, the system framework sets the voltage to this specific value; if they are unequal, the voltage defaults to the PMIC's hardware power-on default voltage.

4. RTC Section

If the RTC function is not required (e.g., for box products), add the node as described above and explicitly set `status = "disabled"`. To enable the RTC, either remove the entire RTC node or set its status to `status = "okay"`.

5. Poweroff Section

```
gpio_poweroff {
        compatible = "gpio-poweroff";
        gpios = <&gpio2 GPIO_D2 GPIO_ACTIVE_HIGH>;
        status = "okay";
};
```

Since the RK816 supports powering down the entire PMIC by pulling the pmic_sleep pin high, this node must be added under the root node. The gpios property is configurable and specifies the pmic_sleep pin.

If this feature is not registered, software-based shutdown via the RK816 can still be used, following the same procedure as for the RK808 and RK818.

4.4 Kernel Configuration

The DTS configuration includes: I2C bus attachment, main device node, RTC, power key, GPIO, regulator, and other components.

```
&pinctrl {
        pmic {
                pmic_int_l: pmic-int-l {
                rockchip,pins =
                        <2 6 RK_FUNC_GPIO &pcfg_pull_up>;       /* gpio2_a6 */
                };
        };
};

&i2c1 {
        status = "okay";
        rk816: pmic@1a {
                compatible = "rockchip,rk816";
                reg = <0x1a>;
                interrupt-parent = <&gpio0>;
                interrupts = <2 IRQ_TYPE_LEVEL_LOW>;
                pinctrl-names = "default";
                pinctrl-0 = <&pmic_int_l>;
                rockchip,system-power-controller;
                wakeup-source;
                gpio-controller;
                #gpio-cells = <2>;
                #clock-cells = <1>;
                clock-output-names = "rk816-clkout1", "rk816-clkout2";
                extcon = <&u2phy>;

                vcc1-supply = <&vcc_sys>;
                vcc2-supply = <&vcc_sys>;
                vcc3-supply = <&vcc_sys>;
                vcc4-supply = <&vcc_sys>;
                vcc5-supply = <&vcc_io>;
                vcc6-supply = <&vcc_sys>;

                gpio {
                        status = "okay";
                };

                pwrkey {
                        status = "okay";
                };

                rtc {
                        status = "okay";
                };

                battery {
                        compatible = "rk816-battery";
                        ocv_table = < 3500 3625 3685 3697 3718 3735 3748
                                      3760 3774 3788 3802 3816 3834 3853
                                      3877 3908 3946 3975 4018 4071 4106>;
```

```
                design_capacity = <2500>;
                design_qmax = <2750>;
                bat_res = <100>;
                max_input_current = <1500>;
                max_chrg_current = <1300>;
                max_chrg_voltage = <4200>;
                sleep_enter_current = <300>;
                sleep_exit_current = <300>;
                sleep_filter_current = <100>;
                power_off_thresd = <3500>;
                zero_algorithm_vol = <3850>;
                max_soc_offset = <60>;
                monitor_sec = <5>;
                virtual_power = <0>;
                power_dc2otg = <0>;
                dc_det_adc = <0>;
        };

        regulators {

                vdd_arm: DCDC_REG1{
                        regulator-name= "vdd_arm";
                        regulator-min-microvolt = <750000>;
                        regulator-max-microvolt = <1500000>;
                        regulator-ramp-delay = <6001>;
                        regulator-initial-mode = <1>;
                        regulator-always-on;
                        regulator-boot-on;
                        regulator-state-mem {
                                regulator-off-in-suspend;
                                regulator-suspend-microvolt = <900000>;
                        };
                };

                vdd_log: DCDC_REG2 {
                        regulator-name= "vdd_logic";
                        regulator-min-microvolt = <750000>;
                        regulator-max-microvolt = <1500000>;
                        regulator-ramp-delay = <6001>;
                        regulator-initial-mode = <1>;
                        regulator-always-on;
                        regulator-boot-on;
                        regulator-state-mem {
                                regulator-on-in-suspend;
                                regulator-suspend-microvolt = <1000000>;
                        };
                };
                vcc_ddr: RK816_DCDC3@2 {
                        ..................
                };
                ..............................
        };
```

```
        };
};
```

1. I2C Attachment

The complete RK816 node must be attached under the corresponding I2C node, with `status = "okay";` configured.

2. Main Section

- Do not modify:

```
compatible = "rockchip,rk816";
reg = <0x1a>;
rockchip,system-power-controller;
wakeup-source;
gpio-controller;
#gpio-cells = <2>;
```

- Modifiable (according to pinctrl rules)

interrupt-parent: Specifies which GPIO the pmic_int belongs to; interrupts: Specifies the pin index and polarity of pmic_int on the GPIO defined in interrupt-parent; pinctrl-names: Do not modify; fixed as "default"; pinctrl-0: References the pmic_int pin defined in pinctrl;

3. rtc, pwrkey, gpio

If these modules are selected in menuconfig but the corresponding Drivers do not need to be enabled in practice, you can add rtc, pwrkey, and gpio nodes in the DTS and explicitly set their status to status = "disabled". This prevents the Drivers from being enabled, although error logs will appear during boot and can be ignored. To enable the Drivers, remove the corresponding nodes or set their status to status = "okay".

4. regulator

- `regulator-compatible`: The name used for matching during Driver registration. Do not modify this value, otherwise the Driver will fail to load.
- `regulator-name`: The power supply name. It is recommended to keep this consistent with the hardware schematic, as this name must be matched when using the `regulator_get` interface.
- `regulator-init-microvolt`: The initialization voltage during the U-Boot stage; this is invalid in the kernel stage.
- `regulator-min-microvolt`: The minimum voltage adjustable during runtime.
- `regulator-max-microvolt`: The maximum voltage adjustable during runtime.
- `regulator-initial-mode`: The DCDC operating mode during runtime, typically configured as 1. 1: force PWM, 2: auto PWM/PFM.
- `regulator-mode`: The DCDC operating mode during sleep, typically configured as 2. 1: force PWM, 2: auto PWM/PFM.
- `regulator-initial-state`: The mode during suspend; this must be configured as 3.
- `regulator-boot-on`: When this property is present, the power supply is enabled upon Driver registration.
- `regulator-always-on`: When this property is present, it indicates that the power supply cannot be disabled during runtime and will be enabled upon Driver registration.
- `regulator-ramp-delay`: The DCDC voltage rise time, fixed at 12500.

- `regulator-on-in-suspend`: Maintains the power-on state during sleep. To disable this power supply during sleep, change this to `regulator-off-in-suspend`.
- `regulator-suspend-microvolt`: The standby voltage when power is maintained during sleep.

4.19 Kernel Configuration

Refer to the 4.4 kernel DTS configuration. Difference: The 4.19 kernel DTS configuration no longer requires the gpio subnode, but other modules still reference and use RK816 pins using the method `gpios = <&rk816 0 GPIO_ACTIVE_LOW>;`.

2.3 Function Interfaces

The following interfaces basically meet daily usage requirements, including regulator enable, disable, voltage setting, and voltage retrieval:

1. Get regulator:

`struct regulator *regulator_get(struct device *dev, const char *id)`

Set dev to NULL by default; id corresponds to the regulator-name property in the DTS.

2. Release regulator

`void regulator_put(struct regulator *regulator)`

3. Enable regulator

`int regulator_enable(struct regulator *regulator)`

4. Disable regulator

`int regulator_disable(struct regulator *regulator)`

5. Get regulator voltage

`int regulator_get_voltage(struct regulator *regulator)`

6. Set regulator voltage

`int regulator_set_voltage(struct regulator *regulator, int min_uV, int max_uV)`

Ensure min_uV = max_uV when passing parameters; this must be guaranteed by the caller.

7. Example

```
struct regulator *rdev_logic;

rdev_logic = regulator_get(NULL, "vdd_logic");                  // Get vdd_logic
regulator_enable(rdev_logic);                                                  // Enable
vdd_logic
regulator_set_voltage(rdev_logic, 1100000, 1100000);    // Set voltage to 1.1V
regulator_disable(rdev_logic);                                                 // Disable
vdd_logic
regulator_put(rdev_logic);                                                            //
Release vdd_logic
```

Note: Kernels 4.4 and 4.19 also provide `devm_`-prefixed regulator interfaces to assist developers in managing requested resources.

3. Debug

3.1 Kernel 3.10

Since the logic involved in PMIC drivers is not complex, with the focus primarily on final register configurations, the common debugging method is to directly inspect RK816 registers via the following node:

`/sys/rk816/rk816_test`

Read register:

`echo r [addr] > /sys/rk816/rk816_test`

Write register:

`echo w [addr] [value] > /sys/rk816/rk816_test`

Example:

`echo r 0x2f > /sys/rk816/rk816_test // Read the value of register 0x2f, which is 0x9b`

```
1|shell@rk3228h:/ # echo r 0x2f >  /sys/rk816/rk816_test
[  283.091704]  [2:              sh:  618] ------zhangqing: get cmd = r
[  283.091766]  [2:              sh:  618] CMD : r 2f
[  283.091914]  [2:              sh:  618]
[  283.091914]  [2:              sh:  618] 2f 9b
```

`echo w 0x2f 0x9c > /sys/rk816/rk816_test // Set the value of Register 0x2f to 0x9c`

It is recommended to perform a read operation after a write operation to verify that the write was successful.

```
shell@rk3228h:/ # echo w  0x2f 0x9c >  /sys/rk816/rk816_test
[  371.974131]  [3:              sh:  618] ------zhangqing: get cmd = w
[  371.974192]  [3:              sh:  618] get value = 9c
[  371.974412]  [3:              sh:  618] 9c   9c
```

3.2 4.4 Kernel

The command format is identical to that of the 3.10 kernel; only the node path differs. The debug node path on the 4.4 kernel is:

`/sys/rk8xx/rk8xx_dbg`

3.3 4.19 Kernel

Refer to the commands for the 4.4 kernel.